

Season 1: How LLMs work

Tokens.

The unit every model reads, writes, and bills in

Everything is counted in tokens

- **Cost:** providers bill per input token and per output token
- **Limits:** context windows and output limits are token counts
- **Latency:** output arrives one token at a time (S1 E1)
- **Fairness:** some languages need far more tokens for the same meaning

A token is a chunk of text, not a word

A tokeniser splits text into pieces from a fixed vocabulary and turns each piece into a number. The model never sees letters or words.

Where it breaks

Counting letters or reversing a word is hard for a model, because it sees chunks, not letters. It is why earlier models miscounted the Rs in "strawberry": the word arrives as a single token, not ten letters.

One sentence, eleven tokens

Input: "My name is Brendan, welcome to my course everyone."

My	name	is	Brendan	,	welcome	to	my
5444	1308	382	128004	11	12591	316	922
course	everyone	.					
4165	6524	13					

Each piece becomes an ID in a vocabulary of about 200,000 entries.

Rarer words split into pieces: "tokenisation" becomes two tokens.

token	isation
6602	6993

Common words like these are one token each, leading space included. Rarer words split into sub-word pieces. Either way, the model only sees the numbers.

Split by o200k_base, OpenAI's current tokeniser. The numbers are real token IDs.

How the vocabulary is built

Byte-pair encoding, used in some form by most current models.

Start with bytes Any text is a sequence of bytes, so no input is ever unknown

Merge frequent pairs The most common neighbouring pair becomes a new token, again and again

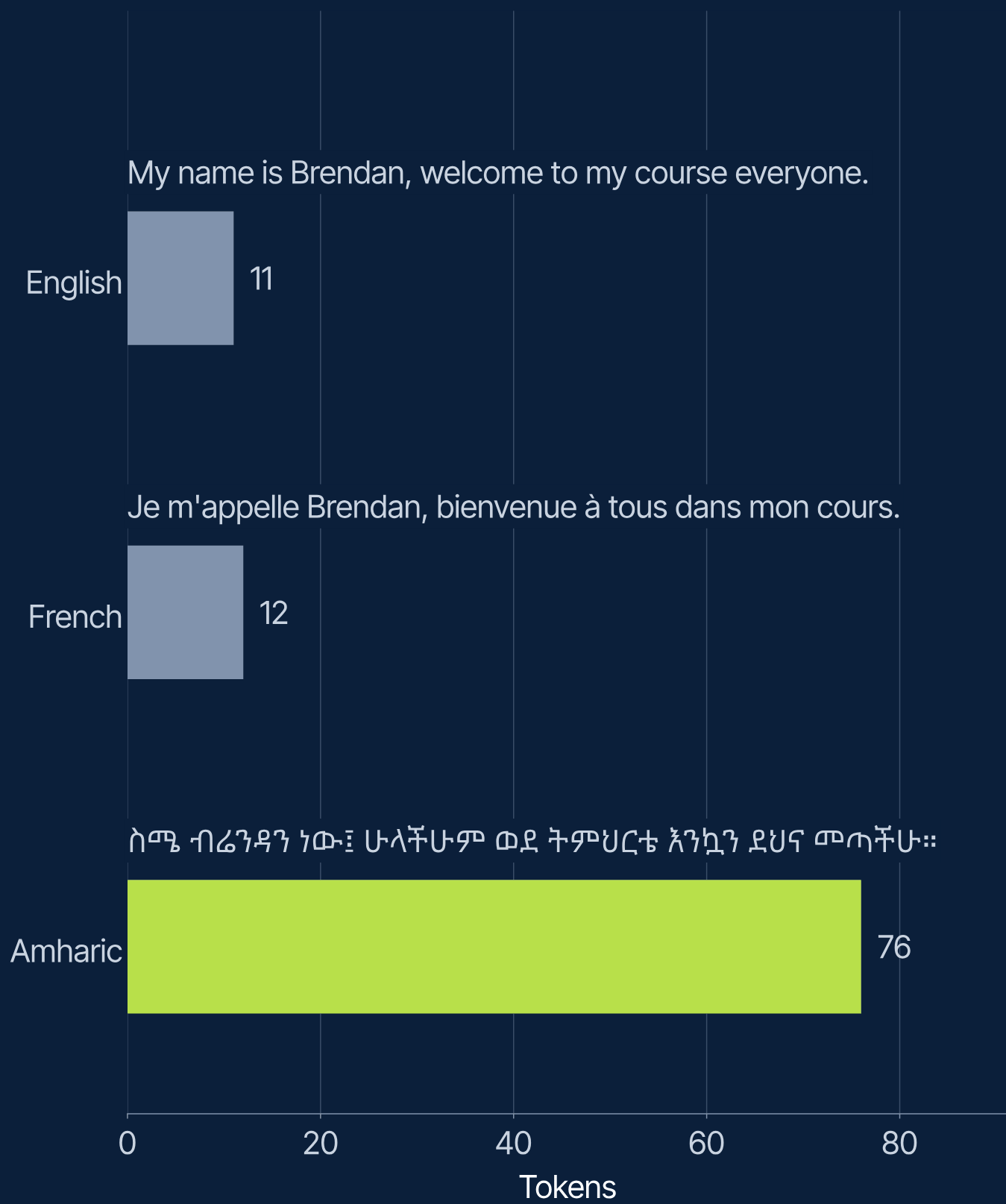
Stop at a fixed size The vocabulary is frozen, and the model is trained on it

The consequence

Common text gets whole-word tokens. Amharic, written in Ge'ez script at three bytes a character, falls back to fragments: 42 characters, 110 bytes, 76 tokens.

The same sentence: 11 tokens in English, 76 in Amharic

Counted with o200k_base. Green marks the costliest language.



The same sentence in each language.
Tokens, counted with o200k_base · One sentence per language

Whose tokeniser you ask changes the bill

The same three sentences, counted on 19 September 2026.

Tokeniser	English	French	Amharic	Amharic vs English
OpenAI o200k_base	11	12	76	6.9x
OpenAI cl100k_base	11	14	101	9.2x
Claude Sonnet 5	18	27	56	3.1x
Gemini 3.6 Flash	12	14	29	2.4x

■ The widest gap: the GPT-4 era tokeniser

o200k_base is current; cl100k_base is from the GPT-4 era. Claude and Gemini counts come from each provider's token counting endpoint; Claude's has 6 message-formatting tokens subtracted.

Count them yourself

No API key, no model call: tokens are counted on your machine.

```
import tiktoken

encoding = tiktoken.get_encoding("o200k_base")

sentences = {
    "English": "My name is Brendan, welcome to my course everyone.",
    "French": "Je m'appelle Brendan, bienvenue à tous dans mon cours.",
    "Amharic": "ስሜ ብሬንዳን ነው፤ ሁላችሁም ወደ ትምህርቴ እንኳን ደህና መጣችሁ።",
}

for language, sentence in sentences.items():
    tokens = encoding.encode(sentence)
    print(f"{language}: {len(tokens)} tokens {sentence}")
```

■ Unlike a model's reply, a token count never varies

[episodes/s1-e2-tokens/count_tokens.py](#) in the [applied-ai-lab](#) repository.

"About four characters a token" holds for English only

- **Code and numbers:** symbols, spacing, and long digit strings split unpredictably
- **Other scripts:** text outside the tokeniser's training mix splits into many small pieces
- **Other providers:** each has its own tokeniser. In S1 E7, Claude Sonnet 5 counted 42% more tokens than GPT-5.6 Terra for the same documents

Measure on your own text, with the model you will use.

History is paid for on every call

A model remembers nothing (S1 E1), so a conversation resends everything so far. Input grows with every turn.



Illustrative sizes. Run `first_call.py` from S1 E1: the third call's input tokens include the whole history.

Tokens you pay for but never see

Reasoning models generate reasoning tokens before they write the answer. You are billed for those tokens as output, but you usually never see them.

```
# Hidden reasoning, already counted in output_tokens  
first.usage.output_tokens_details.reasoning_tokens
```

Where it breaks

A short visible answer can carry a large bill. Budget on output tokens, not on answer length.

From `first_call.py` in the `applied-ai-lab` repository. S1 E10 measures when reasoning is worth it.

What breaks in production

- **Estimating from word counts:** cost forecasts miss for code, numbers, and other languages
- **Silent truncation:** a response that hits the output token limit stops mid-answer and can still look complete. Check the finish reason on every call: "length" (OpenAI) or "max_tokens" (Anthropic) means it was cut off.
- **Growing history:** long conversations get slower and more expensive with every turn
- **Uneven service:** users in some languages pay more and get less context

Price per token is not price per task

What a feature costs depends on how many tokens each task uses: history, documents, hidden reasoning, and language.

Ask your team

What does one typical task cost in tokens, and how does that change for our users in other languages?

Decision rule

Budget, limit, and price in tokens, measured on your own traffic in every language you serve.

Next: S1 E3 Embeddings

How models turn tokens into meaning they can compare.

Where this fits

The series map

S1 LLMs	S2 Retrieval	S3 Strategy
S4 Adapting	S5 Agents	S6 Economics
S7 Data	S8 Evaluation	S9 AI teams
S10 Reliability	S11 Security	S12 Delivery
S13 Performance	S14 Resilience	S15 Governance

☐ Leadership season

 Follow	linkedin.com/in/brendan-kelly-irl
 Episode guide	brendankellyai.github.io
 Code	github.com/BrendanKellyAI/applied-ai-lab